

Module 7

Topics: Fun with SAS functions!

Files to download: [mock_data.sas7bdat](#), [bonus7.xlsx](#)

SAS functions can add a lot to your DATA step programming. This lab will work through several examples of SAS functions that can save you from writing a lot of unnecessary code. This is just a small sample of SAS functions – there are of course many more that we don't have time to cover. Many of the examples in this lab are based on examples from the book below – if you would like to learn more about SAS functions or just have a handy reference guide I recommend this book!

Cody, Ron. 2010. *SAS® Functions by Example, Second Edition*. Cary, NC: SAS Institute Inc..

You can also search for functions/more info on <http://support.sas.com/documentation>.

7.1 - COMPRESS

COMPRESS(<source><, chars><, modifiers>)

- source is the value from which you want to remove characters
- chars is a list of characters you want to remove from *source*
- modifiers are optional and let you specify character classes
 - o a (upper and lower case letters)
 - o d (numerals/digits)
 - o i (ignores case)
 - o k (keep listed characters instead of removing them)
 - o s (space)
 - o p (punctuation)

Sometimes when there aren't strict validation/regex rules on a survey item, people can enter their response in various formats. Phone numbers are always tricky to standardize since there are several appropriate phone number formats. In this example, we will standardize phone numbers that were entered several different ways.

```
data phone;
  input Phone $15.;
  Phone1 = compress(Phone);
  Phone2 = compress(Phone, '(-) ');
  Phone3 = compress(Phone, , 'kd');
datalines;
(708)245-3350
(202) 555-76 99
555-555-1212
;
```

Take a look at the output from the program above.

	Phone	Phone1	Phone2	Phone3
1	(708)245-3350	(708)245-3350	7082453350	7082453350
2	(202) 555-76 99	(202)555-7699	2025557699	2025557699
3	555-555-1212	555-555-1212	5555551212	5555551212

COMPRESS did the following:

- In Phone1, the spaces were removed
- In Phone2, it removed the parentheses, spaces and dashes (note that since the 3rd observation didn't have parentheses it looks the same)
- In Phone3, k and d (with 2 commas) tell SAS to keep only the numeric digits (note that Phone2 would have kept any extraneous characters in it but there weren't any in this example)

7.2 - SUBSTR

SUBSTR(matrix, position<, length>)

- matrix is the input string
- position is the starting position where you want to extract your substring
- length is the length of the string you want to extract (if left blank it will extract the string up to the last non-blank character)

In this example, we will use SUBSTR to extract the state code and ID field from a string. The state code starts in position 3 and has a length of 2 and the ID field starts in position 5 and has different lengths.

Note that when we say "position" we are talking about the order of the characters.

```

                XYNJ1234
Position # :   12345678

data letters_numbers;
    input id $9.;
    length State $ 2;
        State = substr(id,3,2);
        Num = input(substr(id,5),4.);
datalines;
XYNY123
XYNJ1234
;
run;
```

Take a look at the output from the program above.

	id	State	Num
1	XYNY123	NY	123
2	XYNJ1234	NJ	1234

SUBSTR did the following:

- Put the 2 character state code starting in position 3 in a new field called State
- Put the id digits starting in position 5 in a new field called Num (note that the INPUT function is also used to convert the id portion from character to numeric)

7.3 - UPCASE, LOWCASE, PROPCASE

UPCASE(expression)

LOWCASE(expression)

PROPCASE(expression)

- UPCASE will make all the letters upper case
- LOWCASE will make all the letters lower case
- PROPCASE will capitalize the first character in each word and set the remaining characters to lower case
 - o The default delimiter for PROPCASE is a space so each word can be capitalized but you can also specify a single quote as a delimiter in case someone has a ' in their name (note the single quote is inside double quotes)

```
data names_cases;
  input Name $15.;
        Upper = upcase(Name);
        Lower = lowercase(Name);
        Proper = propcase(Name, " '");
datalines;
jOhN DOE
D'Angelo
;
run;
```

Take a look at the output from the program above.

	Name	Upper	Lower	Proper
1	jOhN DOE	JOHN DOE	john doe	John Doe
2	D'Angelo	D'ANGELO	d'angelo	D'Angelo

7.4 - SCAN

SCAN(expression, n [, delimiters])

- expression is the string you want to parse
- n is which "word" you want to extract
- delimiters are optional but good to specify

In the example below, we will parse last name from a full name string.

AEPI537D

```
data presidents;
    length Last_Name $ 15;
    input @1 Name $20.;
        Last_Name = scan(Name, -1, ' ');
datalines;
Donald Trump
Barack Obama
George W. Bush
Bill Clinton
George H. W. Bush
Ronald Reagan
Jimmy Carter
Gerald Ford
Richard Nixon
Lyndon B. Johnson
;
run;
```

Take a look at the output from the program above.

	Last_Name	Name
1	Trump	Donald Trump
2	Obama	Barack Obama
3	Bush	George W. Bush
4	Clinton	Bill Clinton
5	Bush	George H. W. Bush
6	Reagan	Ronald Reagan
7	Carter	Jimmy Carter
8	Ford	Gerald Ford
9	Nixon	Richard Nixon
10	Johnson	Lyndon B. Johnson

SCAN did the following:

- Parsed the last name and put it in a new variable called Last_Name
- Note: the -1 tells SAS to scan from right to left, so when a name is recorded as first-middle-last, you will always get the last name

Let's do one more example of SCAN. In this example we have a comma separated list of vegetables and we want to separate them into separate variables.

```
data veggies;
    input veg_list $30.;
datalines;
beet,carrot,broccoli
beet,celery
beet,pepper,tomato
;
run;

data veggies2;
    set veggies;
```

```
Veg1= SCAN(veg_list,1,',');
Veg2= SCAN(veg_list,2,',');
Veg3= SCAN(veg_list,3,',');
run;
```

Take a look at the output from the program above.

	veg_list	Veg1	Veg2	Veg3
1	beet,carrot,broccoli	beet	carrot	broccoli
2	beet,celery	beet	celery	
3	beet,pepper,tomato	beet	pepper	tomato

Here SCAN made is 3 new variables and using the comma as a delimiter, it was able to place the value of each vegetable in its own field. Note that Veg3 is blank for row 2 because a 3rd vegetable was not listed.

7.5 - FIND, SUBSTR, LOWCASE

FIND(string,substring<,modifiers> <,startpos>) or FIND(string,substring<,startpos> <,modifiers>)
 SUBSTR(matrix, position<, length>)
 LOWCASE(expression)

We've already used SUBSTR and LOWCASE but we haven't used FIND. This function can find the position of a specific character. In the example below, we will use several functions to parse the domain from a list of email addresses and place it in a separate variable. This might be useful if you want to know how many people in your list have Gmail, Hotmail, etc.

```
data svu_emails;
    input email $50.;
datalines;
olivia.benson@gmail.com
ice-t@yahoo.com
elliott.stabler@gmail.com
donaldcragen@aol.com
doctor_huang@hotmail.com
docwarner@gmail.com
jmunch911@aol.com
amanda.rollins@comcast.net
;
run;

data svu_emails2;
    set svu_emails;
    email_domain1=find(email, "@");
    email_truncate=substr(email, email_domain1+1);
    email_domain2=find(email_truncate, ".");
    email_domain_name=lowercase(substr(email_truncate, 1, email_domain2-1));
    drop email_domain1 email_domain2 email_truncate;
run;
```

Take a look at the output from the program above.

	email	email_domain_name
1	olivia.benson@gmail.com	gmail
2	ice-t@yahoo.com	yahoo
3	elliott.stabler@gmail.com	gmail
4	donaldcragen@aol.com	aol
5	doctor_huang@hotmail.com	hotmail
6	docwamer@gmail.com	gmail
7	jmunch911@aol.com	aol
8	amanda.rollins@comcast.net	comcast

In this example:

- FIND looked for the position of the @ symbol in the email field
- SUBSTR truncated the email from the @ on
- FIND looked for the position of the dot (.) between the domain and the com
- SUBSTR extracted the domain by specifying the location before and after it
- LOWCASE made these all uniform
- We also dropped the intermediate variables we no longer needed

7.6 - TRANWRD

TRANWRD(source-expression, target-expression, replacement-expression)

- source-expression is the input string
- target-expression is the find string
- replace-expression is the replace string

In this example, we will standardize some address fields by converting Road, Street, and Avenue, to their abbreviations.

```
data add_convert;
  input @1 address $20. ;
      Address = tranwrd(Address, 'Street', 'St. ');
      Address = tranwrd(Address, 'Avenue', 'Ave. ');
      Address = tranwrd(Address, 'Road', 'Rd. ');
datalines;
89 Lazy Brook Road
123 River Rd.
9731 South Avenue
12 Main Street
;
run;
```

Take a look at the output from the program above.

AEPI537D

	address
1	89 Lazy Brook Rd.
2	123 River Rd.
3	9731 South Ave.
4	12 Main St.

TRANWRD did the following:

- Looked for the words Road, Avenue, and Street and converted them to the abbreviated version
- Note that we replaced the original variable Address with another variable called Address so we ended up overwriting the same field

7.7 - MDY, MONTH, WEEKDAY, DAY, YEAR, YRDIF

In the example below, you will see various date functions.

- MDY will take separate day, month, and year values and create a date
- WEEKDAY will return the numeric day of the week from a date (1=Sunday, 2=Monday, etc.)
- DAY will return the day from a date (1 to 31)
- MONTH will return the month from a date (1 to 12)
- YEAR will return the year from a date
- YRDIF computes the number of years between 2 dates

```
data date_funcs;
  input (Date1 Date2)(:mmddyy10.) M D Y;
    SAS_Date = MDY(M, D, Y);
    WeekDay = weekday(Date1);
    MonthDay = day(Date1);
    Year = year(Date1);
    Age = yrdif(Date1, Date2);
  format Date: mmddyy10.;
datalines;
10/21/1955 10/21/2012 6 15 2011
;
run;
```

Take a look at the output from the program above.

SAS_Date	WeekDay	MonthDay	Year	Age
18793	6	21	1955	57

7.8 - SMALLEST, LARGEST

SMALLEST(k, expression, [, ...expression])

LARGEST(k, expression, [, ...expression])

These functions can be used to get the nth smallest or largest number in a list of values. The k specifies if you want the smallest, second smallest, etc. (or largest) from the list. Both functions ignore missing values.

```
data small_large;
  input x1-x5;
    S1 = smallest(1, of x1-x5);
    S2 = smallest(2, of x1-x5);
    L1 = largest(1, of x1-x5);
    L2 = largest(2, of x1-x5);
datalines;
7 2 . 6 4
10 . 2 8 9
;
run;
```

Take a look at the output from the program above.

	x1	x2	x3	x4	x5	S1	S2	L1	L2
1	7	2	.	6	4	2	4	7	6
2	10	.	2	8	9	2	8	10	9

In the output above, S1 took the smallest (first smallest) value from the x1 to x5 list, while S2 took the second smallest. L1 and L2 did something similar for the largest and second largest values.

7.9 - TRIMN, STRIP

TRIMN(expression)

STRIP(expression)

CATS(item-1[, ...item-n])

- TRIMN removes trailing blanks
- STRIP removes leading and trailing blanks
- CATS removes leading and trailing blanks and concatenates strings

In the example below, strings are concatenated in several different ways to demonstrate the differences.

```
data concat_string;
  length Concat $ 8;
  One = '  ABC  ';
  Two = 'XYZ';
  One_two = ':' || One || Two || ':';
  Trim = ':' || trimn(One) || Two || ':';
  Strip = ':' || strip(One) || strip(Two) || ':';
  Concat = cats(':', One, Two, ':');
  put one_two= / Trim= / Strip= /
      Concat=;
run;
```

Take a look at the output from the program above.

	Concat	One	Two	One_two	Trim	Strip
1	:ABCXYZ:	ABC	XYZ	: ABC XYZ:	: ABCXYZ:	:ABCXYZ:

Without the TRIMN and STRIP functions, the variable keeps all the blanks when the strings are concatenated. The TRIMN variable has no blanks between the ABC and XYZ and the STRIP variable has no blanks at all. CATS removes leading and trailing blanks and concatenates all in one.

7.10 - CEIL, FLOOR, INT, ROUND

CEIL(argument)

FLOOR(argument)

INT(expression)

ROUND(expression [, rounding-unit])

- CEIL returns the smallest integer that is greater than or equal to the argument
- FLOOR returns the largest integer that is less than or equal to the argument
- INT returns the integer value
- ROUND rounds the first argument to the nearest multiple of the second argument or nearest integer if no second argument
- All of these are fuzzed to avoid unexpected floating-point results

The example below shows the result for each of these on the same numbers: 7.2, 7.8, -7.2, and -7.8.

```
data truncate;
  input x @@;
    ceil = ceil(x);
    floor = floor(x);
    int = int(x);
    round = round(x);
datalines;
7.2 7.8 -7.2 -7.8
;
run;
```

Take a look at the output from the program above.

	x	ceil	floor	int	round
1	7.2	8	7	7	7
2	7.8	8	7	7	8
3	-7.2	-7	-8	-7	-7
4	-7.8	-7	-8	-7	-8

7.11 - ZIPFIPS, ZIPNAME, ZIPNAMEL, ZIPSTATE

SAS has some built in functions for working with US zip codes.

- ZIPFIPS returns the FIPS code from a 5 digit zip code
- ZIPNAME returns the state name in all caps from a 5 digit zip code
- ZIPNAMEL returns the state name in propcase from a 5 digit zip code
- ZIPSTATE returns the 2 letter state abbreviation from a 5 digit zip code

```
data zips_states;
  input zip @@;
  fips = zipfips(zip);
  state_caps = zipname(zip);
  state_mixed = zipnamel(zip);
  state_abbre = zipstate(zip);
  format zip z5.;
datalines;
30322 33018 98101 02801 73301
;
run;
```

Take a look at the output from the program above.

	zip	fips	state_caps	state_mixed	state_abbre
1	30322	13	GEORGIA	Georgia	GA
2	33018	12	FLORIDA	Florida	FL
3	98101	53	WASHINGTON	Washington	WA
4	02801	44	RHODE ISLAND	Rhode Island	RI
5	73301	48	TEXAS	Texas	TX

7.12 - COUNTC, CATS

COUNTC(string, charlist <,modifiers>)

CATS(item-1[, ...item-n])

- COUNTC counts the number of characters in a string that appear or do not appear in a list of characters
- CATS removes leading and trailing blanks and concatenates strings

In the example below, COUNTC is being used to count the number of Y's (upper or lower case - the 'i' is telling SAS to ignore case when it count the Y's) collected in a survey and uses CATS to concatenate all the survey responses into a single string.

```
data Survey;
  input (Q1-Q5)($1.);
  Num = countc(cats(of Q1-Q5), 'y', 'i');
datalines;
yynnY
```

AEPI537D

```
nnnnn
;
run;
```

Take a look at the output from the program above.

	Q1	Q2	Q3	Q4	Q5	Num
1	y	y	n	n	Y	3
2	n	n	n	n	n	0

7.13 - ARRAY

Arrays are useful to process a series of variable with similar inputs.

- DIM takes the name of the array and returns the number of elements in the array so you don't have to count them
- An * can be used in place of the number of array elements

In the example below, we will be using arrays to work on ALL the numeric and ALL the character variables in the data set.

```
data convert;
  input (A B C)($) x1-x3 y z;
  array nums[*] _numeric_;
  array chars[*] _character_;
  do i = 1 to dim(nums);
    if nums[i]=999 then nums[i]=.;
  end;
  do i = 1 to dim(chars);
    chars[i] = propcase(chars[i], " ");
  end;
  drop i;
datalines;
RON jOhN mary 1 2 999 3 999
;
run;
```

Take a look at the output from the program above.

	A	B	C	x1	x2	x3	y	z
1	Ron	John	Mary	1	2	.	3	.

In this program, the array "chars" went through all the character variables and converted them to PROPCASE, and the array "nums" went through all the numeric variables and set values of 999 to missing. DIM was used to avoid having to count the number of each variable type in advance.

7.14 – Bonus Example – REVERSE, INPUT, SUBSTR, STRIP

Let's say you were given an excel file where the Last_Visit variable was a long character string of different lengths, and you need to isolate the section that contains the date and ultimately have a month/year variable you can use to categorize people in a chart or table.

Import the bonus7.xlsx file to SAS and take a look at the columns.

```
PROC IMPORT OUT= WORK.bonus7 DATAFILE= "H:\AEPI537D\bonus7.xlsx"
  DBMS=xlsx REPLACE;
  SHEET="Sheet1";
  GETNAMES=YES;
RUN;
```

The Last_Visit column is just one big text string with various lengths. We can't take the first X number of characters because in each row the date portion begins in a different position.

	Study_Id	Last_Visit
1	1001	Extended Visit 04/13/2018
2	1002	FOLLOW UP 08/27/2018
3	1003	Extended Visit 09/04/2018
4	1004	FOLLOW UP 09/26/2018
5	1005	FOLLOW UP 10/22/2018
6	1006	Extended Visit 05/03/2018
7	1007	FOLLOW UP 10/08/2018
8	1008	Walk-In Visit 11/05/2018
9	1009	Walk-In Visit 10/03/2018
10	1010	FOLLOW UP 10/18/2018
11	1011	FOLLOW UP 11/28/2018
12	1012	FOLLOW UP 10/18/2018
13	1013	Routine 10/13/2018
14	1014	FOLLOW UP 09/12/2018
15	1015	FOLLOW UP 11/19/2018
16	1016	Extended Visit 08/01/2018
17	1017	Routine 08/17/2018
18	1018	FOLLOW UP 09/26/2018
19	1019	Walk-In Visit 20 10/18/2018
20	1020	Extended Visit 06/08/2018

One creative solution for isolating the date might be to right justify the contents of the column and count backwards to get only the last 10 characters (there are 10 characters in a date field formatted as MM/DD/YYYY including the forward slashes).

The following code can be used to reverse the order of the string in the Last_Visit column, and put the first 10 characters of the reversed string into a new field.

```
data bonus7_a;
  set bonus7;
  newvar=substr(reverse(strip>Last_Visit)),1,10);
run;
```

	Study_Id	Last_Visit	newvar
1	1001	Extended Visit 04/13/2018	8102/31/40
2	1002	FOLLOW UP 08/27/2018	8102/72/80
3	1003	Extended Visit 09/04/2018	8102/40/90
4	1004	FOLLOW UP 09/26/2018	8102/62/90
5	1005	FOLLOW UP 10/22/2018	8102/22/01

Of course, now the order of the string is still reversed and must be read backwards to make sense. We could simply reverse again to get it back in the right order.

```
data bonus7_b;
  set bonus7_a;
  newvar2=left(reverse(newvar));
run;
```

	Study_Id	Last_Visit	newvar	newvar2
1	1001	Extended Visit 04/13/2018	8102/31/40	04/13/2018
2	1002	FOLLOW UP 08/27/2018	8102/72/80	08/27/2018
3	1003	Extended Visit 09/04/2018	8102/40/90	09/04/2018
4	1004	FOLLOW UP 09/26/2018	8102/62/90	09/26/2018
5	1005	FOLLOW UP 10/22/2018	8102/22/01	10/22/2018

We need SAS to understand that this remaining text in newvar2 is a date, so we can use the INPUT statement to convert it.

```
data bonus7_c;
  set bonus7_b;
  newdate=input(newvar2,mmddyy10.);
run;
```

	Study_Id	Last_Visit	newvar	newvar2	newdate
1	1001	Extended Visit	04/13/2018	8102/31/40	04/13/2018
2	1002	FOLLOW UP	08/27/2018	8102/72/80	08/27/2018
3	1003	Extended Visit	09/04/2018	8102/40/90	09/04/2018
4	1004	FOLLOW UP	09/26/2018	8102/62/90	09/26/2018
5	1005	FOLLOW UP	10/22/2018	8102/22/01	10/22/2018

If the ultimate goal is to have just a date field in a month/year format to use as a category, we could modify the previous step to include a different date format.

```
data bonus7_d;
  set bonus7_b;
  newdate=input(newvar2,mmddyy10.);
  format newdate monyy7.;
run;
```

	Study_Id	Last_Visit	newvar	newvar2	newdate
1	1001	Extended Visit	04/13/2018	8102/31/40	04/13/2018
2	1002	FOLLOW UP	08/27/2018	8102/72/80	08/27/2018
3	1003	Extended Visit	09/04/2018	8102/40/90	09/04/2018
4	1004	FOLLOW UP	09/26/2018	8102/62/90	09/26/2018
5	1005	FOLLOW UP	10/22/2018	8102/22/01	10/22/2018

MOD 7: USE SAS COMMENTS TO NUMBER YOUR WORK.

Using the mock_data.sas7bdat dataset provided, do the following in a SINGLE data step (create one new dataset from mock_data where you complete all the functions below). You can run this data step by adding one line at a time to check and make sure each item works, or you can use separate data steps and then consolidate to one after you have tested each item.

1. Replace the street address variable (ADDRESS) with one that converts any instance of the word "Street" with the abbreviation "St."
2. Create a new variable called SUPLNM and store only the last name of the supervisor from the variable SUPERVISOR.
3. Create a new variable called STATE and store the state abbreviation using the zip code variable (ZIPCODE).
4. Create a new variable called THISYEAR and store the year value from the date field called DATE.
5. Round the number stored in the VALUE field to the nearest whole number and put it in a new variable called VALRND.
6. Create a new phone number variable called PHONEFIX from the original called PHONE_NUM that stores the phone number as only numbers, with no dashes or parentheses.
7. Create a variable called LRGNUM that stores the largest value from the variables SCORE1, SCORE2, SCORE3, SCORE4, and SCORE5.
8. Split the values in the variable COLORS into 3 separate variables called C1, C2, and C3.
9. Create a variable called CODE_NEW that stores the value of the variable CODE without any leading or trailing spaces.
10. Convert the CODE_NEW variable to all lowercase and store it in a new variable called CODE_NEW2.